# Objects and class.

## Defining a class

A class is a blue print used to create objects that defines data and behavior of for objects.

-> It groups related fields and methods in a single unit.

syntax:

```
class class-name
{
  // variable
  // methods
}
```

Ex:-

```
class Student
{
  string name; // variable
  int age;     // variable
  void display() // method
  {
    system.out.println("Name :"+name);
    system.out.println("Age:"+age);
  }
}
```

## Adding variables.

When defining a class in java, we can add variables to store the data or properties of an object these varibles are called instance variable

Ex:-

```
Public class Student
{
  string name;
  int age;
  double marks;
  public static void main (String args[])
  {
    Student S1 = new Student();
    S1.name = "Rahul";
    S1.age = 22;
    S1.marks = 85.5;
    system.out.println(S1.name)
    system.out.println(S1.age)
    System.out.println(S1.marks);
  }
}
```

## Adding methods.

A method is a block of code inside a class that performs a specific task or defines the behavior of an objects

syntax:

```
class class_name
{
  returntype method-name()
  {
    // statements
  }
}
```

Ex:-

```
public class student
{
  string name;
  int age;
  public static void main display()
  {
    system.out.println("Name:" + name);
    system.out.println("age:"+age);
  }
  public static void main(string[] args)
  {
    student S1 = new student();
    S1.name = "Rahul";
    S1.age = 20;
    S1.display();
  }
}
```

## Creating objects

After defining a class, we can create objects from that class.

-> An object is an instance of a class it represents a real world entity and it contains own data and we can use methods define inside the class.

syntax:-

```
class name  object_name = new class_name ();
```

Ex:-

```
student s1 = new student ();
```

Here, student -> classname
s1 -> object_name
new -> creating object
student(); -> calls the class constructor

Example program

```
class student {
    string name;
     int age;
void display (){
system.out. println ("Name :"+name);
system.out. println ("age :"+ age);
  }
public static void main (string args[])
    { student s1 = new student ();
      name = "Rahul";
      age = 25;
      display();
    }
}
```

output
Name: Rahul
age: 25

## Accessing class members

Class memobers are the elements defined inside a class such as
-> variables -> store data
=> methods -> Performs actions
-> after creating an object, we can access its members using the dot(.) operator.

syntax:
for variable
objectname.membername;
for method
objectname.methodname;

Ex:-

```
class student
{
  string name;
  int age;
void display ()
{
system.out.println ("Name :"+name);
system.out.println ("Aage:"+ name);
}
public static void main (string args[])
    { student s1 = new student ();
      student s2 = new student();
      s1.name = "Rahul";
      s1.age = 25;
      s2.name = "kavya";
      s2.age = 25;
      s1.display();
      s2.display();
    }
}
```

## Constructor

A constructor in Java is a special memobers that is called automatically when object is created.
-> It is used to set default or user define value for the objects attributes.

* Rules:
-> constructor & class name should be same
-> It doesnot have a return type not even void.
-> It can acept paramebers to initialized object properties

Ex:

```
class student
{
 string name;
// constructor
 student (string name)
 { this.name = name;
 }
 void display ()
 { system.out.println ("name :"+name);
 }
```

```
public static void main (String [] args)
{ student s1 = new student ("vishnu");
  s1.display();
 }
}
```

Type of constructor
1) default constructor
2) parameterized constructor

## Finalize() methods

It is in java is called by the barbage collector just before an object is destroyed.
→ It allows the object to perform a clean up activity.
→ It is defined in the object class and avilable to all java class.
→ It can be overiden to perform clean up activity

Syntax:
```
@override
protected void finalize()
{
  // clean up code
}
```

Ex:-
```
class Demo
{ @override
  protected void finalize()
  { system.out.println("object is destroyed");
  }
  public static void main (String args[])
  { Demo d1 = new Demo();
    d1 = null;
    system.gc();
  }
}
```

Override finalize() method.
The finalize() method, which is present in the object class as an empty implemention in over class clean up activities are there then we have to overried this med to define owes clean up activity.

## Method overloding.

It allows multiple methods with the same name but different parameters with in the same class.
→ It is also called as compile time polymorphism.
→ It allows a method to perform simler operations with different type or number of inputs.

Ex:-
```
public class overloadDemo
{
  static int my-sum (int a, int b)
  { return a+b;
  }
  static int my-sum (int a, int b, int c)
  { return a+b+c;
  }
  public static void main (String args[])
  { System.out.println ("My sum with 2 integer parameters:"
                        + my-sum (4,6));
    system.out.println ("My sum with 3 integer parameters:"
                        + my-sum (4,6,7));
  }
}
```

## Method Overriding

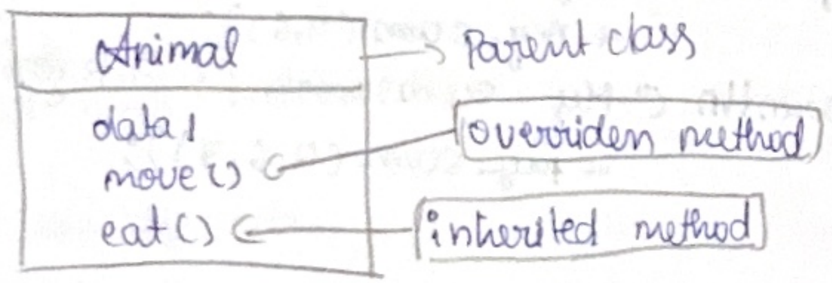
It is in java allows a sub class to provide a speatic implementation of a method that is already define in its parent class
→ It is one of the key feature of runn time polymorphism in
→ static, final and private methods cannot be overreadan.

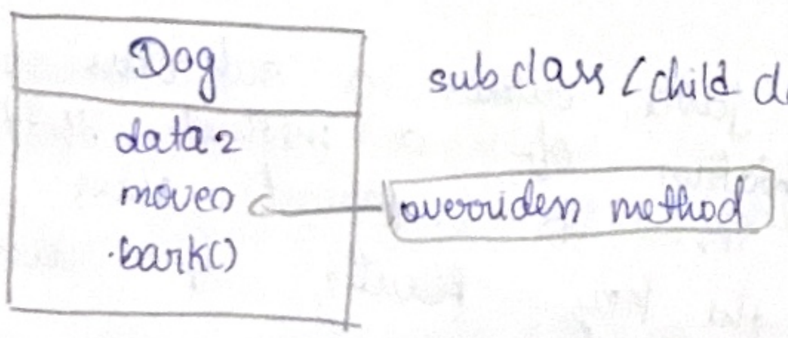
Ex:-
```
class animal {
  void move()
  { system.out.println ("Animal is moving");
  }
  void eat()
  { system.out.println ("Animal is eating");
  }
}
```

```
class dog extends Animal
{ @Override void move()
  { system.out.println ("Dog is running");
  }
  void bark()
  { system.out.println ("Dog is Barking");
  }
}
public class Main
{ public static void main (String args[])
  { Dog d = new Dog();
    d.move();
    d.eat();
    d.bark();
  }
}
```

## Rules

-> name, parameters and return type must match the parent method
-> static methods can not be overridden
-> The @override annotation catches the mistakes.




## Final variable.

In Java a final variable is a variable whose value can be assigned only once. Once initialized it can't be changed.

Syntax:

```
final datatype variable_name;
```

* final variable can be declared without initialization but it must be initialized exactly once before it is used.

Ex:-

```
public class student
{ final int rollno;
  student (int rollno)
  { this.rollno = rollno;
  }
  public static void main (String[] args)
  { student S = new student (101);
    system.out.println (S.rollno);
  }
}
```

Final variable can be initialized
→ At the time of declaration
→ inside a constructor (for instance variable)
→ inside a static block (for static final variables)

final is commonly used to create constants, prevent accidental modification and improve code readability.

## Final methods

final method in java is a method that can't be overridden by a sub class
The final keyword is used when we want to prevent a methods implementation from being changed through method overriding.

Ex:-
```
class Animal
{ final void sound()
system.out.println("Animal makes a sound");
}
}
class dog extends animal
{ void sound()
  { system.out.println("dog barks");
  }
}
public class sample
{ public static void main (string[] args)
  { dog d = new dog();
    d.sound();
  }
}
```

Why use final methods?

final methods are used to prevent method overriding protect important or critical implemention
- maintain consistent behaviour in sub class
- improve program security and reliability.

final class

final class in java is a class that can't be inherited (extended) by c

→ final keyword is used with a class when we want to prevent inheritance

Ex:-
```
final class student
{ void display()
  { system.out.println("Student class");
  }
}
public class main
{ public static void main (string args[])
  { student s = new student();
    s.display();
  }
}
```

Abstract class in Java

An abstract class in Java that is declear using the abstract keyword, it is used as a base class for other classes and can not be instasiated directly.

Ex:-
```
Abstract class Animal {
    abstract void sound(){
      void eat()
        {
        system.out.println("Animal eats");
        }
}
class dog extends Animal {
        void sound(){
        System.out.println("Dog barks");
        }
}
public class Sample {
public static void main (sting() args)
{ Dog D = new Dog();
  D.sound();
  D.eat();
}
}
```

Abstract method in Java

An abstract method is a method that has no body. it declared using the abstract keyword it most be defined in the subclass.

Syntax:- abstract void sound(); // No body.

Visiability control in Java (Access modifiers)

Visiability control in Java are also called as access modifiers they determin where a class, a variable,

method or constructor can be accessed

Java has four level of access controls

1. Public:- Public member can accessed form any where in the program.
2. protected:- This member can be a accessed within the same package in subclass involoving subclass in other packages.
3. default:- when no accessed modifier is speaified it is called default access
4. private:- This member can be accessed only within the same class

Exmple :-

```
class student
{  private int a=10;
           int b=20; //default
   protected int c=30;
   public int d = 40;
}
```

| Access modifiers | same class | same package | subclass different package | other package |
|---|---|---|---|---|
| private | ✓ | x | x | x |
| default | ✓ | ✓ | x | x |
| protected | ✓ | ✓ | ✓ | x |
| Public | ✓ | ✓ | ✓ | ✓ |

## strings

strings in Java is an object use to store a sequence of characters enclosed in double codes.

-> Such as "Hellow" or "Java programming"

-> The important ~~o~~ improtenent proprtiy of Java string is that they are immutable, meaning once string object is created, it's value cann't be ~~cone~~ changed.

-> Any ~~oper~~ operation that appears to modify a string actualy creats a new string.

Common string methods in Java.

1) length()
-> It returns the length of the string.
Ex: "Java".length() -> 4

2) charAt(index)
-> It returns the character of the specified index
Exa: "Java".charAt(1) -> 'a'

3) toUpperCase()
-> It convert the given string into upper case.
Ex:- "java".toUpperCase() -> "JAVA"

4) toLowerCase():
-> It convert the given string ento lower case.
Ex: "JAVA".toLowerCase -> "java"

5) Substring(start)
-> It returns a part of the string.
Ex:- "Java".substring(2) -> va

6) Substring(start, end)
-> It returns a position of the string
Ex: "Java".Substring(0, 3)
-> Java

7) equals()
-> It compares a string contents
Ex:- "abc".equals("abc") -> true

String contatenation

Strings can be joined are combained using the + operator is called string contatenation.

```
Exa: string str1 = "Java";
     string str2 = "Programming";
     string .Name = str1 + " " + str2;
```

System.out.println(Name);

Output: Java Programming.

## String buffer class in Java.

It is peer class of string. while string creates a string of fixed lenght, String buffer creates strings of flexible lenght and that can be modified in terms of both lenght and contents.

→ we can insert characters and substring in the middle of the string or ~~append~~ append another string at the end of a string

Exp:

```
import java.lang.string;
class StringBufferDemo
{ public static void main (String args[])
  { StringBuffer SB = New . StringBuffer ("this is my college")
    System.out.println ("This String SB is:"+SB);
    System.out.println ("The length of the string is:"+
                        SB.length());
    System.out.println ("The character at the index 6 is:"+
                        SB.charAt(6));
    SB.setCharAt(3,'x');
    System.out.println ("Appending:" +SB.append
                        ("in davangere"));
  }
}
```

## Vectors

It is a class in java that can be used to create dynamic arrays which can holds objects of any type and of any number.

→ It is used to implement the list, random access, cloneable, serializable interfaces.

Syntax:

```
Vector v = new Vector();
```

Example:

```
import java.lang.*;
import java.~~vecto~~util.vector;
class vector
{ public static void main (String args[])
  {
    Vector V = new Vector();
    V.addElement ("one");
    V.addElement ("two");
    V.addElement ("three");
    V.insertElementAt("zero",0);
    V.insertElementAt("OOps",3);
    V.insertElementAt("Four",5);
    System.out.println ("Vector size:"+V.size());
    System.out.println ("~~Vector~~ The elements of the vector
                        are:");
  }
}
```

## wrapper classes in Java

Java wrapper class are the class that converts primitive data types into objects, they are part of the java.lang package.

| primitive type | wrapper class |
|---|---|
| byte | Byte |
| short | Short |
| int | Int |
| long | Long |
| float | Float |
| double | Double |
| char | Char |
| boolean | BoOlean |

Ex:-
```
class WrapperClass
{public static void main (String args[])
{ int a=10;
  Integer Obj=a;
  int b = obj
  System.out.println("Primitive: "+a);
  System.out.println ("Wrapper object:"+obj);
  System.out.println ("Unboxed primitive:"+b);
}
}
```

## Static members / ~~static class~~

In java a static data member also called a static variable (or) class variable is a variable that belongs to the class itself not to individual objects.

→ This means there is only one copy of the variable shared by all object of the class.

Ex:-
```
class student
{ String name;
  static string college = "winners college";
  Student (String n)
  {
    name = n;
  }
  void display ()
  { System.out.println (name + "studies at" + college);
  }
  }
  public class sample
  { public static void main(String args[])
  {
    student s1 = new student ("Chandhar vaa");
    student s2 = new student ("Akshara");
    s1.display();
    s2.display();
  } }
```

## Static class

They are also special type of nested class. a top level class cannot be declared static. only a class defined inside another class can be static

Example.
```
class outer
{
 static int x=100
static class Inner.
{
  void show()
  { system.out.println ("value of x =" + x);
  }
}
public class sample
{ public static void main (string args[])
{ Outer.Inner obj = new Outer.Inner ();
  obj.show();
}
}
```